Informe Laboratorio 3

Sección 3

Edicson Solar Salinas
e-mail: edicson.solar@mail.udp.cl

Octubre de 2026

Índice

1. Descripción de actividades **2**

2. Desarrollo de actividades **2**

2.1. Identifica el algoritmo hash utilizado para las contraseñas al momento del registro en el sitio web 2

2.2. Identifica el algoritmo hash utilizado para las contraseñas al momento de iniciar sesión 4

2.3. Genera el hash de la contraseña desde la consola del navegador 5

2.4. Intercepta el tráfico login con Burp Suite 7

2.5. Realizar el intento de login por medio del hash 8

2.6. Identifica las políticas de privacidad o seguridad 10

2.7. Conclusiones y comentarios sobre la seguridad de MEGA 12

Referencias **12**

Anexo **12**

1. Descripción de actividades

En este laboratorio revisé cómo MEGA procesa una contraseña en el navegador y si el valor derivado para el inicio de sesión podía reutilizarse. Elegí MEGA porque publica el código de su cliente web y un whitepaper del protocolo. Usé una cuenta creada solo para esta prueba, sin archivos personales.

Las actividades fueron las siguientes:

- Identificar el procedimiento criptográfico usado durante el registro y relacionarlo con la solicitud observada en DevTools.
- Identificar el procedimiento de inicio de sesión, incluidas las solicitudes `us0` y `us`.
- Reproducir en la consola del navegador la derivación de la credencial de autenticación y compararla con el valor capturado.
- Observar el inicio de sesión con Burp Suite.
- Hacer un único control con contraseña incorrecta y reutilizar una sola vez la credencial correcta mediante Repeater.
- Revisar las políticas oficiales sobre contraseña, cifrado, Recovery Key y seguridad del dispositivo.
- Cerrar el análisis con cuatro conclusiones sobre la seguridad de MEGA.

Guardé localmente los HAR y las capturas originales. No probé cuentas ajenas ni intenté acceder a archivos con la respuesta de Repeater. Antes de incorporar las imágenes al informe oculté el correo, la sal, `uh`, las claves y los identificadores de sesión.

2. Desarrollo de actividades

2.1. Identifica el algoritmo hash utilizado para las contraseñas al momento del registro en el sitio web

Durante el registro guardé el tráfico de DevTools *Network* en un HAR. Al revisarlo observé una solicitud `up` y, después, `uc2`. Centré el análisis criptográfico en `uc2`, porque allí aparecían `crv`, `hak`, `k` y `v:2`.

El HAR permitió observar los campos enviados, pero no revelaba cómo se calculaban. El whitepaper describe el protocolo MEGA v2 y `js/security.js` muestra cómo lo implementa el cliente oficial [1, págs. 16 a 17][2]. Durante el registro, el cliente genera un *Client Random Value* de 128 bits y construye una sal de 256 bits de la forma indicada en la ecuación 1. La letra P completa 200 caracteres junto con la cadena `mega.nz`; después se concatena el valor aleatorio del cliente [1, págs. 16 a 17].

$$S = \text{SHA-256}\left(\texttt{mega.nz} \parallel P^{200-|\texttt{mega.nz}|} \parallel CRV\right) \tag{1}$$

Con esa sal y la contraseña, el navegador calcula 256 bits mediante:

$$DK = \text{PBKDF2-HMAC-SHA-512}(\textit{password}, S, 100\,000, 256\ \textit{bits}). \tag{2}$$

La salida no se usa como una sola pieza. Los primeros 128 bits forman la *Derived Encryption Key*, empleada para cifrar con AES-ECB la Master Key aleatoria de 128 bits. Los segundos 128 bits forman la *Derived Authentication Key*. Sobre esta última se calcula SHA-256 y se conservan los primeros 128 bits como *Hashed Authentication Key* o `hak`. El servidor recibe el valor aleatorio del cliente, la Master Key cifrada y `hak`; no recibe la contraseña en texto claro ni la clave de cifrado derivada.

La figura 1 muestra la solicitud real de registro. La acción `uc2` contiene `crv`, `hak`, `k` y `v:2`. El campo `k` corresponde a la Master Key cifrada; `crv` permite volver a calcular la sal y `hak` es el verificador descrito. Antes de incorporar la captura al informe oculté el correo, los datos de cuenta y los valores criptográficos sensibles; conservé visibles los nombres de los campos, la acción `uc2` y la versión 2.

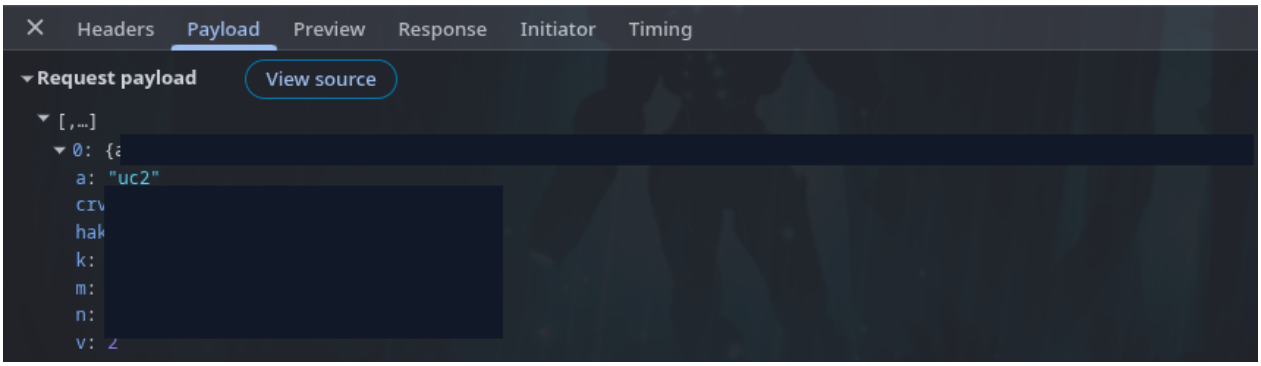


Figura 1: Solicitud `uc2` observada durante el registro de la cuenta de laboratorio.

Los rectángulos oscuros corresponden a valores sensibles censurados; los nombres de campos y el resultado técnico se conservaron.

Campo	Interpretación utilizada en el análisis
`crv`	Valor aleatorio del cliente, de 128 bits, utilizado para reconstruir la sal.
`k`	Master Key de 128 bits cifrada con la primera mitad de la clave derivada.
`hak`	Primeros 128 bits de SHA-256 aplicado a la mitad de autenticación.
`v`	Versión del protocolo; la captura muestra el valor 2.

Tabla 1: Correspondencia entre campos del registro y la especificación de MEGA v2.

2.2. Identifica el algoritmo hash utilizado para las contraseñas al momento de iniciar sesión

Después del registro activé la cuenta desde otro navegador, fuera del proxy, para no capturar el buzón ni el token de confirmación. Como el navegador de Burp todavía mostraba la pantalla de confirmación, eliminé únicamente el almacenamiento local de MEGA y volví a abrir el formulario de inicio de sesión.

Antes de enviar el formulario limpié *Network*, activé *Keep log* y mantuve *Intercept* desactivado en Burp. Ingresé las credenciales correctas de la cuenta de laboratorio y envié el formulario una sola vez. En el HAR exportado identifiqué primero `us0`, cuya respuesta contenía `s` y `v:2`, y después `us`, cuya solicitud contenía `user` y `uh`.

Relacioné esos campos con el flujo descrito en las páginas 19 y 20 del whitepaper y en `js/security.js`: el cliente decodifica `s` desde Base64URL y repite PBKDF2-HMAC-SHA-512 con 100 000 iteraciones y 256 bits de salida [1, págs. 19 a 20][2]. La figura 2 conserva la estructura de la respuesta sin publicar la sal.

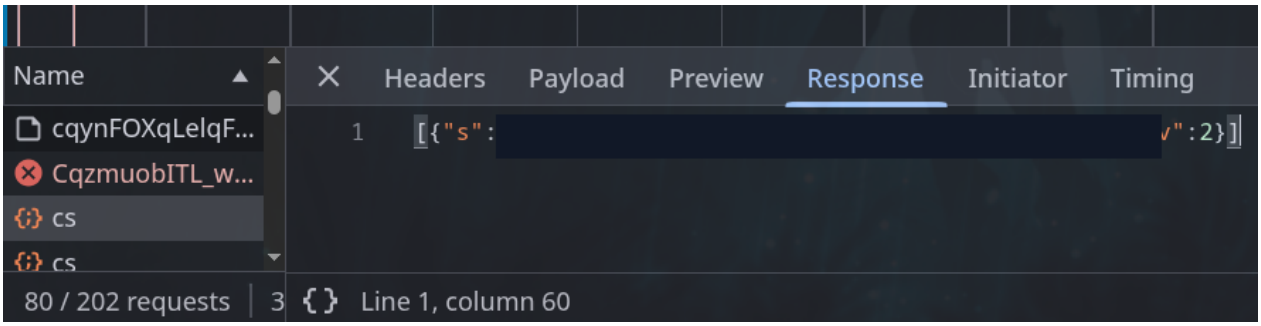


Figura 2: Respuesta de `us0`: sal censurada y versión 2 del protocolo.

La primera mitad de la salida vuelve a ser la clave que permite descifrar la Master Key. La segunda mitad, de 128 bits, es la clave de autenticación. El cliente la codifica en Base64URL y la envía como `uh` en la acción `us`, junto con el correo. La figura 3 muestra que la solicitud incluyó esos campos y no incluyó la contraseña en texto claro.

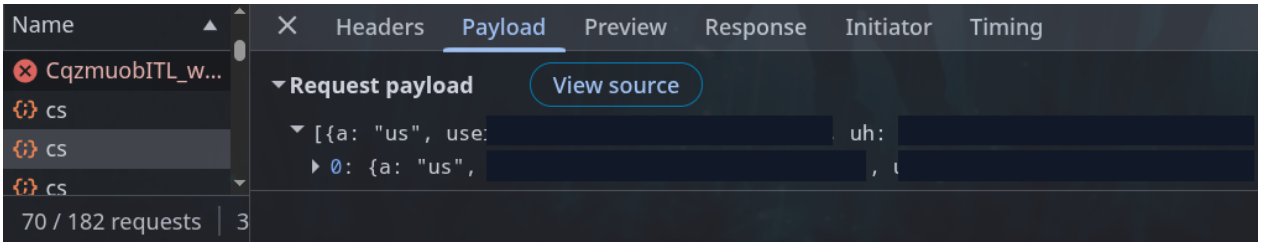


Figura 3: Solicitud `us` del inicio de sesión; correo y `uh` censurados.

Según el whitepaper, el servidor aplica SHA-256 a la clave de autenticación recibida y compara los primeros 128 bits con el `hak` almacenado [1, pág. 20]. En el HAR, la respuesta a `us` devolvió HTTP 200 y los campos `ach`, `csid`, `k`, `privk` y `u`. Esto confirmó que la API aceptó

el inicio de sesión y que `uh` corresponde a la clave de autenticación derivada, no al verificador almacenado. En la actividad siguiente comprobé esa relación con el valor capturado.

2.3. Genera el hash de la contraseña desde la consola del navegador

Para comprobar el cálculo usé un verificador en JavaScript basado en las páginas 19 y 20 del whitepaper y en `js/security.js` [1, págs. 19 a 20] [2]. El verificador reproduce el cálculo de `uh` con Web Crypto. El archivo completo está en `scripts/A3_verificar_pbkdf2_login.js` dentro del repositorio del anexo. Antes de cargar los datos ejecuté `node --check` para descartar errores de sintaxis.

Para comprobar el valor seguí estos pasos:

1. En la captura del inicio de sesión correcto abrí la respuesta de `us0` y tomé su campo `s`. Después abrí la solicitud `us` del mismo flujo y tomé su campo `uh`. No incorporé ninguno de esos valores al informe.

2. Abrí la pestaña *Console* de DevTools en el navegador integrado de Burp y pegué allí el contenido completo del archivo.

3. El código mostró un cuadro con tres campos enmascarados. Allí ingresé la contraseña de la cuenta de laboratorio, `s` y `uh`.

4. Al confirmar, el código decodificó `s` y `uh` desde Base64URL, comprobó que tuvieran 32 y 16 bytes, respectivamente, y codificó la contraseña en UTF-8.

5. Importó la contraseña como material PBKDF2 y derivó 256 bits con HMAC-SHA-512, la sal capturada y 100 000 iteraciones.

6. Tomó los bytes 16 a 31 de la salida, los codificó en Base64URL y los comparó con el `uh` capturado. La consola mostró únicamente los parámetros generales y el resultado de la comparación.

El extracto siguiente contiene la derivación y la comparación de `uh`. El archivo completo del repositorio también incluye el cuadro de entrada, las conversiones, las validaciones de longitud y la limpieza de las variables locales.

Listing 1: Extracto literal del código ejecutado para derivar y comparar `uh`.

```
passwordBytes = new TextEncoder().encode(secrets.password);
const keyMaterial = await window.crypto.subtle.importKey(
    'raw',
    passwordBytes,
    'PBKDF2',
    false,
    ['deriveBits']
);

const derivedBits = await window.crypto.subtle.deriveBits(
    {
        name: 'PBKDF2',
        hash: 'SHA-512',
        salt: saltBytes,
        iterations: 100000
    },
    keyMaterial,
    256
);

derivedKeyBytes = new Uint8Array(derivedBits);
const calculatedUh = bytesToBase64Url(derivedKeyBytes.subarray(16, 32));
const expectedUh = secrets.expectedUh.replace(/=+$/u, '');
const matches = calculatedUh === expectedUh;
```

La consola mostró `COINCIDE`: los segundos 128 bits calculados localmente, después de codificarlos en Base64URL, eran iguales al `uh` capturado. La figura 4 registra ese resultado y los parámetros generales en la pestaña *Console*. Como el verificador no realiza llamadas de red, esta ejecución no produjo otro inicio de sesión ni una solicitud adicional a MEGA.



Figura 4: Resultado de la reproducción local en la consola del navegador.

2.4. Intercepta el tráfico login con Burp Suite

Configuré el navegador para pasar su tráfico por Burp Suite y mantuve *Intercept* desactivado mientras observaba *Proxy / HTTP history*. El registro seleccionado en la figura 5 es un `POST /cs` hacia `g.api.mega.co.nz`. El cuerpo contiene la acción `us` y los campos `user` y `uh`; el estado mostrado es HTTP 200.

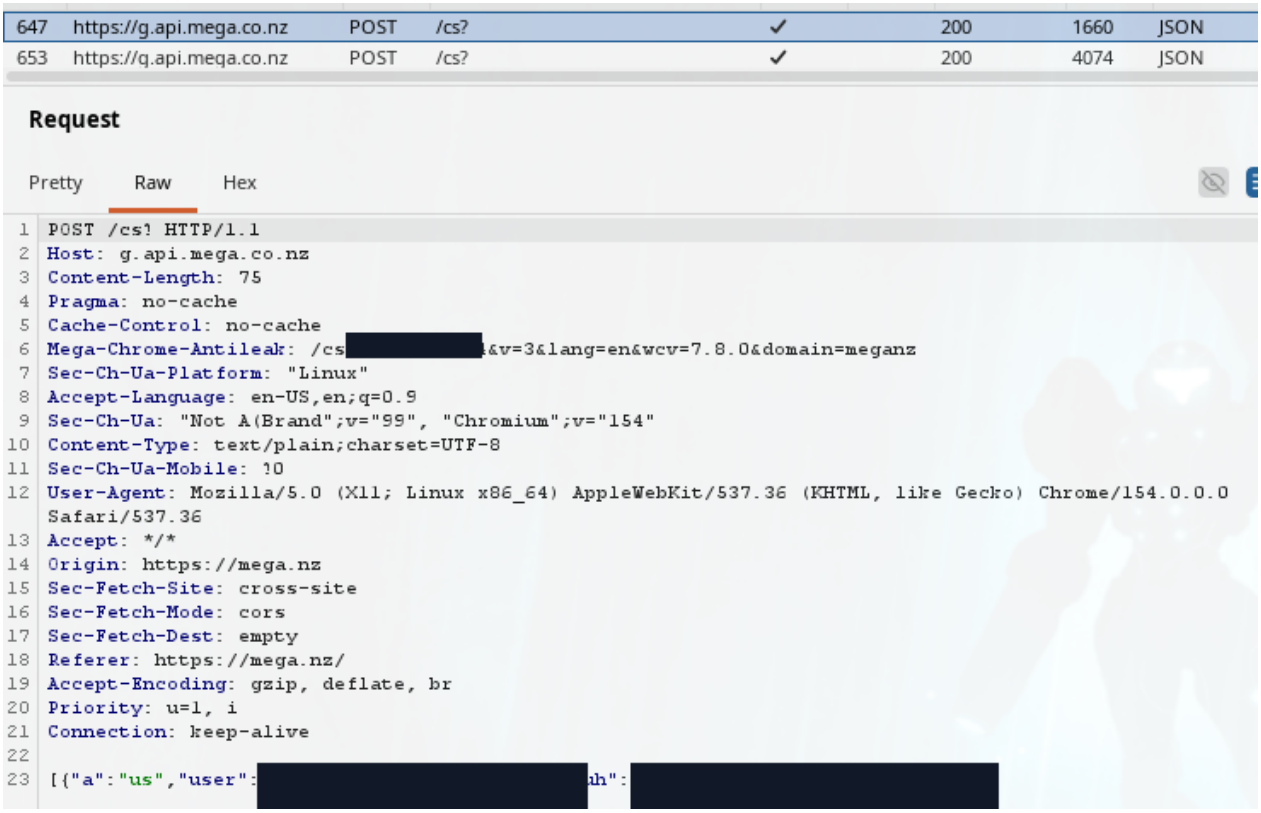


Figura 5: Solicitud de inicio de sesión observada en Burp Suite.

Los rectángulos oscuros corresponden a valores sensibles censurados; los nombres de campos y el resultado técnico se conservaron.

El navegador de laboratorio confiaba en el certificado local de Burp, por lo que el proxy pudo leer el cuerpo. Fuera del proxy, la API utiliza TLS [1, pág. 15]. Como `uh` participa directamente en la autenticación, lo oculté en las capturas y mantuve los archivos originales fuera del repositorio.

2.5. Realizar el intento de login por medio del hash

Primero realicé un solo intento mediante el formulario con una contraseña deliberadamente incorrecta. MEGA mostró `Invalid email address or password`, como se observa en la figura 6. El campo de contraseña quedó vacío en la captura y el correo fue censurado.

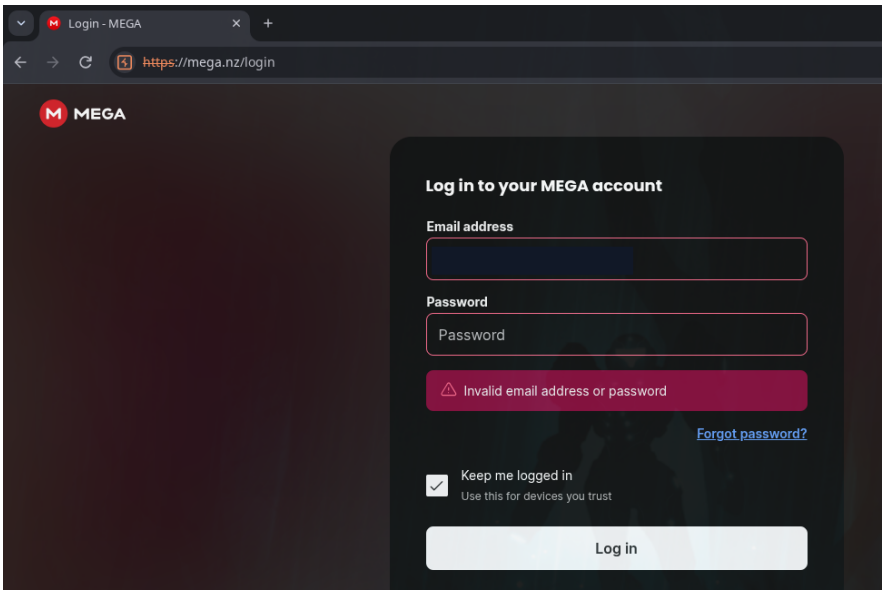


Figura 6: Rechazo visible del único intento con contraseña incorrecta.

El HAR contiene una entrada `us` con HTTP 402 y cuerpo vacío, seguida 19 segundos después por otra entrada `us` con HTTP 200 y cuerpo `[-9]`. No atribuí una causa a la respuesta vacía. El resultado interpretable `-9` concuerda con el rechazo mostrado por la interfaz y evidencia que el estado HTTP 200, por sí solo, no representa un inicio de sesión correcto.

Antes del intento incorrecto había copiado a Repeater la solicitud `us` del inicio de sesión correcto, sin ejecutarla. Después del rechazo la envié una sola vez, sin cambiar `uh`. La API respondió HTTP 200 con un objeto que contenía `csid`, `k`, `privk`, `u` y `ach`. La figura 7 muestra el método, el host, el estado y el inicio del objeto; los valores de autenticación, las claves cifradas y los identificadores quedaron cubiertos por completo.

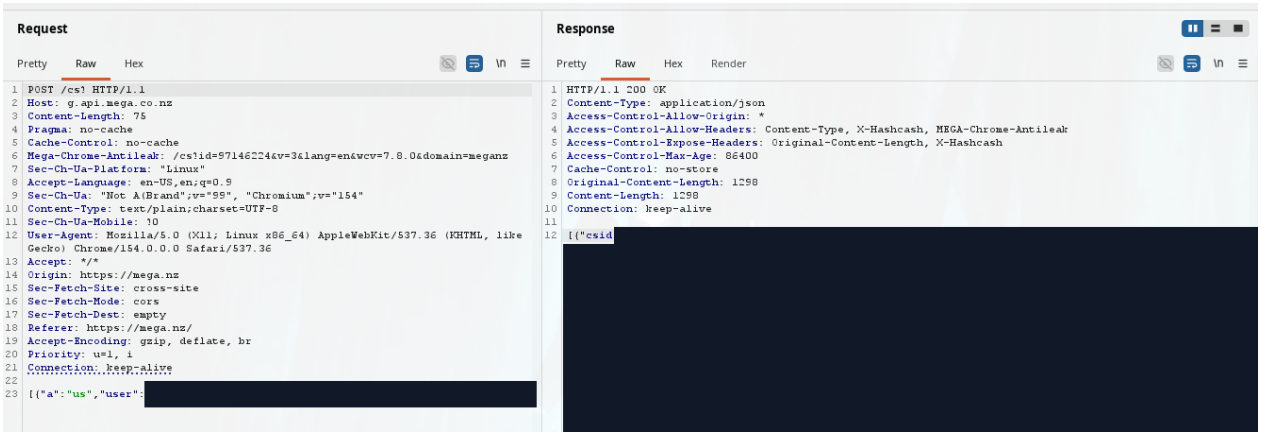


Figura 7: Reutilización controlada del `uh` correcto en Repeater y respuesta de la API.

Control	**Resultado interpretable**	**Qué se comprobó**
Contraseña incorrecta en el formulario	API `-9` y rechazo visible	La credencial derivada desde la contraseña incorrecta no fue aceptada.
`uh` correcto reutilizado una vez	HTTP 200 y objeto de autenticación	La API aceptó el material correcto reutilizado y devolvió material cifrado de inicio de sesión.

Tabla 2: Comparación del control negativo y la reutilización en Repeater.

La API aceptó `uh` y devolvió el objeto de autenticación. Detuve la prueba ahí: no importé `csid`, no abrí una sesión web ni accedí a archivos. Además, `uh` corresponde solo a la mitad de autenticación; para descifrar la Master Key también se necesita la primera mitad derivada. El servidor almacena `hak`, no `uh` [1, pág. 20], por lo que capturar `uh` en el navegador representa un riesgo distinto a obtener la base de datos.

2.6. Identifica las políticas de privacidad o seguridad

Revisé la política de privacidad vigente y los términos generales oficiales, además del whitepaper. La política recomienda contraseñas fuertes y únicas, autenticación de dos factores, actualizaciones y medidas de protección del dispositivo. También advierte que MEGA no solicita la contraseña por correo, que la reutilización de contraseñas aumenta el riesgo de toma de cuenta y que, sin una Recovery Key conservada, olvidar la contraseña puede implicar perder el acceso a los datos [3].

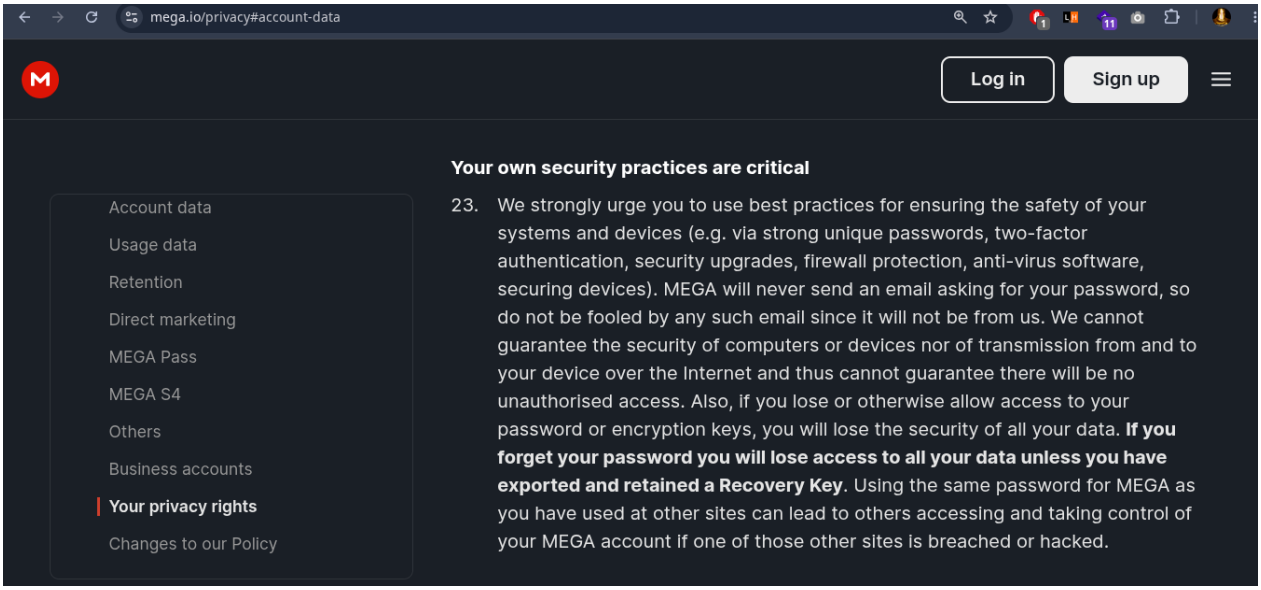


Figura 8: Advertencias de seguridad publicadas en la política de privacidad de MEGA.

El numeral 10 de los términos generales, mostrado en la figura 9, describe el cifrado controlado por el usuario o UCE. Indica que contraseña y Recovery Key deben mantenerse confidenciales, que no deben compartirse las claves salvo que se quiera conceder acceso y que MEGA almacena el correo, pero no la contraseña [4]. Esto coincide con el diseño descrito en el whitepaper: el servidor conserva un verificador derivado y material cifrado, no la contraseña necesaria para descifrar la Master Key.

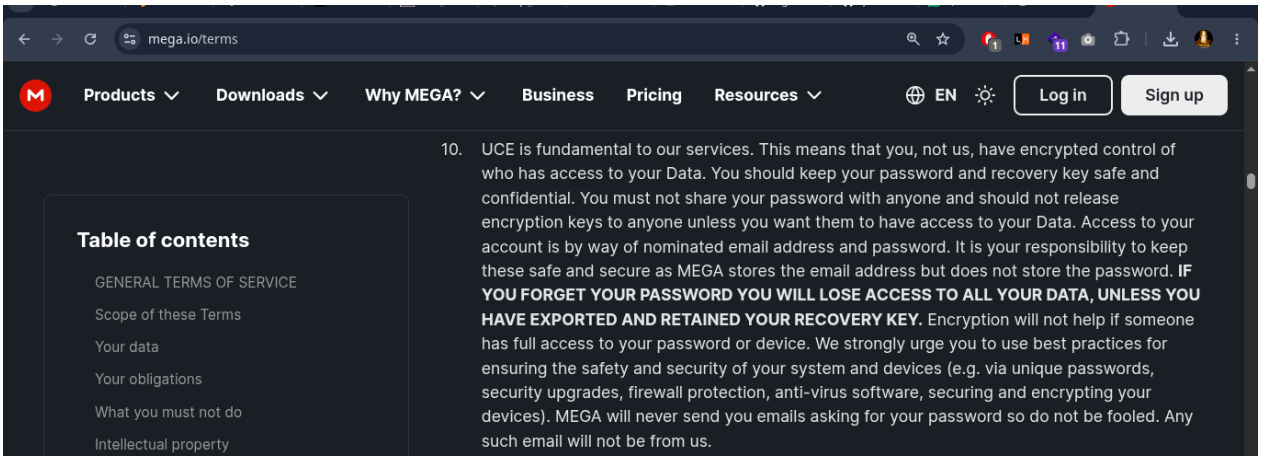


Figura 9: Numeral 10 de los términos generales de MEGA.

El cifrado de extremo a extremo no protege un navegador o equipo comprometido. El malware, el acceso físico o el control del navegador pueden capturar los datos antes del cifrado o después de la derivación [1, pág. 14].

2.7. Conclusiones y comentarios sobre la seguridad de MEGA

1. En el registro y el inicio de sesión comprobé que MEGA v2 no envía la contraseña en texto claro a la API. El cliente deriva 256 bits con PBKDF2-HMAC-SHA-512 y separa el resultado en una mitad para cifrado y otra para autenticación. Por eso `uh` no es la contraseña ni un hash simple de ella: son los segundos 128 bits codificados en Base64URL. La diferencia es importante porque una mitad protege la Master Key y la otra autentica la cuenta.

2. Me llamó la atención que la derivación ocurriera completamente en el navegador y que mi cálculo coincidiera con el `uh` capturado. Ingresé la sal de `us0`, la contraseña y el valor observado; la consola solo mostró los parámetros y el resultado. Así pude contrastar el whitepaper y el código oficial con el tráfico de mi propia cuenta.

3. DevTools fue la parte que más me gustó porque pude seguir `us0` y `us`, revisar sus campos y guardar el HAR. Burp Suite lo usé mucho menos y casi solo para la comprobación en Repeater. La contraseña incorrecta terminó en el código API `-9`, mientras que reutilizar una vez el `uh` correcto devolvió un objeto de autenticación. Detuve la prueba en esa respuesta y no abrí una sesión web ni accedí a archivos.

4. La separación entre cifrado y autenticación me parece acertada. Lo que más me preocupó fue comprobar que un `uh` válido se comporta como una credencial frente a la API. Por eso protegería el navegador y el dispositivo, usaría una contraseña única, activaría 2FA y guardaría la Recovery Key por separado.

Referencias

[1] MEGA Limited. *MEGA Security Whitepaper*, tercera edición, junio de 2022. `https://mega.nz/SecurityWhitepaper.pdf`.

[2] MEGA Limited. *Webclient: implementación oficial de seguridad*. `https://github.com/meganz/webclient/blob/master/js/security.js`. Consultado el 25 de octubre de 2026.

[3] MEGA Limited. *Privacy and Data Policy*. `https://mega.io/privacy`. Consultado el 25 de octubre de 2026.

[4] MEGA Limited. *Terms of Service*. `https://mega.io/terms`. Consultado el 25 de octubre de 2026.

Anexo

- Repositorio